CALIBO AI ACADEMY

Phase 1 — Mini Use Case 01

Retail Sales Pulse — From Analysis to Pipeline

Modules: Python Libraries (EDA) & Data Engineering & Enterprise Systems | Industry: Retail / E-commerce

1. A Note Before You Begin

There is a quote I carry with me and want to share with you before you start this use case:

> *"The day you understand the value of the work you do, is the day you add more value to the work you do."*

This use case has two parts, and that is intentional. The first part asks you to think like an analyst — explore data, find patterns, and tell a business story. The second part asks you to think like an engineer — make that analysis reliable, repeatable, and ready for the real world.

The two skills together are what an AI practitioner brings that neither a pure data analyst nor a pure software engineer can match on their own. By the end of this use case, you will have done both.

2. Use Case Overview

Field	Details
Title	Retail Sales Pulse — From Analysis to Pipeline
Industry	Retail / E-commerce
Phase & Modules	Phase 1 — Python Libraries (EDA) + Data Engineering & Enterprise Systems
Assignment type	Formative — Mini Use Case (two-part)
Part 1 tools	Python · Jupyter Notebook · Pandas · NumPy · Matplotlib · Seaborn
Part 2 tools	Python · Jupyter Notebook · Pandas · JSON · requests (simulated)
Submission	One Jupyter Notebook covering both parts + 1-page written insight summary
Dataset	MUC01_Retail_Sales_Dataset.csv · MUC01_Weekly_Sales_API_Response.json · MUC01_Weekly_Sales_API_

files	Corrupt.json
Deadline	To be confirmed by your trainer

3. Business Context

A regional retail chain operating 12 stores across Andhra Pradesh — Vijayawada, Guntur, Rajahmundry, Tirupati, Nellore, and Kakinada — has six months of transaction data covering January to June 2026. The chain sells across five product categories: Electronics, Apparel, Grocery, Home & Kitchen, and Personal Care.

The Head of Retail Operations needs two things from you:

What she needs	Why it matters
A clear picture of what the data says right now	She is allocating Q3 promotional spend and needs evidence, not gut feel
A way to get this picture automatically every week	She does not want to wait for a data team to re-run an analysis manually each Monday

> This is the natural progression of real data work: first you explore and understand, then you build the machinery that keeps the understanding current. Part 1 does the first. Part 2 does the second.

4. Files Provided

Three files are shared alongside this document. Place all three in the same folder as your Jupyter Notebook before you begin.

File	Used in	What it contains
MUC01_Retail_Sales_Dataset.csv	Part 1	Six months of transaction data — ~108,000 rows, Jan–Jun 2026. 12 stores, 5 categories.
MUC01_Weekly_Sales_API_Response.json	Part 2	A simulated API response — the last 7 days of sales data (Jun 24–30) structured as a JSON payload, mimicking a real retail data API.
MUC01_Weekly_Sales_API_Corrupt.json	Part 2	The same JSON payload with 3 intentional data quality issues introduced. Used for the validation exercise.

> Local setup: All work runs on your own laptop. Install the required libraries with: pip install pandas numpy matplotlib seaborn jupyter To start: open a terminal, navigate to your working folder, and run: jupyter notebook Create one notebook for both parts. Name it [YourName]_MUC01_Notebook.ipynb

PART 1 — Retail Sales Pulse: Exploratory Data Analysis

Load the MUC01_Retail_Sales_Dataset.csv file and work through the six steps below in sequence. Each step has two parts — the code, and a markdown cell in plain English explaining what you did and what you found. The markdown cells are not optional; they are half the assignment.

5.1 Load and inspect the data

Load the dataset and get oriented. Check the shape, column types, and whether there are any null values. Run a .describe() summary and write 2 observations from it — not what the numbers are, but what they might mean for the business.

Column reference:
transaction_id · date · store_id · store_city · category · product_name · units_sold · unit_price · revenue · discount_pct

5.2 Revenue by store

Calculate total revenue per store for the full six months and produce a horizontal bar chart ranked from highest to lowest. Use colour to visually distinguish the top three and bottom three stores from the rest. Follow the chart with 2–3 sentences on what stands out and what it might signal to the operations team.

5.3 Category performance over time

Group revenue by category and month, then plot a line chart showing each category's monthly trend — all five on one chart. Identify which category has shown the steepest decline and which has been most stable. Write 2 sentences of business interpretation: not just "Electronics declined" but what that might mean and what question it raises.

5.4 Day-of-week and monthly patterns

Extract the day of week from the date column and plot average daily revenue by day. Then plot total revenue by month. Identify the best and worst performing day and month. In your markdown, answer: if you were advising the operations head on when to run a flash promotion, what would you say based on this data?

5.5 Anomaly identification

Calculate weekly revenue per store. Flag any store-week combinations where revenue falls more than 2 standard deviations below that store's own average weekly revenue. List the flagged combinations and for each one, suggest one possible business reason — supply disruption, local event, pricing issue, competitor activity. Practising reasoned hypothesis formation is the point.

5.6 Your business recommendation

Close Part 1 with a markdown cell titled "Business Recommendation." In 3–5 sentences, state your single most important finding, the one action you would recommend the operations head take, and what additional data you would want before being fully confident. Be specific — name a store, a category, a number.

PART 2 — Automating the Pulse: Data Pipeline & Validation

The operations head liked your analysis. She now wants it refreshed every Monday morning automatically rather than waiting for someone to re-run it. Your job in Part 2 is to build the pipeline that makes that possible.

> You are simulating what a real data engineering workflow looks like: ingest data from an API, validate it before processing, transform it into something useful, and produce a summary output. The JSON files provided are structured to behave like a real retail data API response — with one clean version and one that has data quality issues deliberately introduced.

5.7 Understand the JSON structure

Load MUC01_Weekly_Sales_API_Response.json using Python's json library and explore its structure. Write a markdown cell answering these three questions:

- What is the top-level structure of this JSON? What keys does it contain?
- Where does the actual transaction data sit within the JSON?
- How would a real retail API use this structure to deliver weekly sales data to a downstream system?

Hint: import json | with open("MUC01_Weekly_Sales_API_Response.json") as f: data = json.load(f)

5.8 Ingest and parse the API response

Extract the transaction records from the JSON and load them into a Pandas DataFrame. Then:

- Convert the date column to datetime format.
- Confirm the record count matches the record_count field in the JSON metadata.
- Print the period covered — from and to dates — from the metadata fields.

- Show the first 5 rows of the resulting DataFrame.

In your markdown cell, explain: why would a production pipeline always check that the record count in the metadata matches the actual records received?

5.9 Validate the data before processing

This is where good data engineering practice separates itself from careless work. Load the corrupt file — MUC01_Weekly_Sales_API_Corrupt.json — and run the same ingestion steps. Then write a validation function that checks for the following and flags any issues found:

Validation check	What to flag
Null values	Any column that contains a null or missing value — flag the row index and column name
Negative revenue	Any transaction where revenue is less than or equal to zero
Invalid store IDs	Any store_id that does not exist in the valid list: S01 through S12

Your validation function should print a clear summary — something a data engineer would actually want to see in a pipeline log:

```
Validation Summary
----------------
Total records checked: 4291
Null values found: 1 (row 12, column: units_sold)
Negative revenue found: 1 (row 5, revenue: -150.0)
Invalid store IDs found: 1 (row 20, store_id: S99)
Status: FAILED — 3 issues detected. Pipeline halted.
```

In your markdown, answer: what should a well-designed pipeline do when it encounters corrupt data — stop completely, skip the bad rows, or fix them automatically? Justify your answer with one sentence.

5.10 Transform and aggregate

Now work with the clean JSON file. After ingesting and validating it, produce the following outputs — each as a named variable or DataFrame:

- **— Total revenue per store for the 7-day period.** weekly_revenue_by_store
- **— Total revenue per category for the 7-day period.** weekly_revenue_by_category
- **— The single highest-revenue store, with its revenue figure.** top_store_this_week
- **— Any category where this week's revenue is more than 20% below its 6-month daily average (use the CSV dataset to compute the baseline).** category_alert

The last one connects Part 1 and Part 2 directly: you need the historical analysis from the CSV to make the weekly pipeline meaningful. That connection is the point.

5.11 Pipeline summary output

Build a function called generate_weekly_summary() that takes the clean JSON file as input and prints a structured weekly report. The output should look something like this:

```
=== RETAIL SALES PULSE — WEEKLY SUMMARY ===
Period: 2026-06-24 to 2026-06-30
Records processed: 4291

Top store this week: S01 — Vijayawada (₹X,XX,XXX)
Bottom store this week: S12 — Kakinada (₹X,XX,XXX)

Category alert: Electronics revenue down 23% vs 6-month baseline
No alert: Apparel, Grocery, Home & Kitchen, Personal Care

Pipeline status: COMPLETED SUCCESSFULLY
```

The actual numbers will come from your data. The format above is the target. In your markdown, explain: what would you add to this summary if it were being emailed to the operations head every Monday morning?

---

6. CBIM Problem Canvas

Complete this canvas before you write a single line of code. It covers the full use case — both parts. If you cannot answer these questions clearly before you start, you are not yet ready to build.

Canvas element	Your answer
Situation (S)	What is the current state? What exists today in terms of data and process?
Complication (C)	What problem or gap exists? What is the operations head unable to do right now?
Question (Q)	What is the specific business question this use case must answer — for both the analysis and the automation?
SSOT	What is the single source of truth? For Part 1 — the historical CSV. For Part 2 — the weekly API response. How do they relate?
Success definition	How will you know this use case was successful? One metric for Part 1, one for Part 2.
Primary stakeholder	Who is this for? What decisions do they need to make, and when?

Submit the completed canvas as a clearly formatted section at the start of your notebook — labelled "CBIM Problem Canvas."

7. Deliverables

The submission is one notebook covering both parts, plus an insight summary. Read the relevant section for your working mode.

If you are working individually

What to submit	Details
Jupyter Notebook	One .ipynb file with CBIM canvas at the top, then Part 1 (Steps 5.1–5.6) and Part 2 (Steps 5.7–5.11) in sequence. Runs end-to-end without errors.
Insight summary	1-page Word document (max 500 words): three findings from Part 1, one pipeline design decision you made in Part 2 and why, and one thing you would do differently with more time.

Criterion	Weight	What good looks like
Part 1 — Code & analysis	30%	All six steps complete, runs without errors, markdown cells explain business meaning not just technical steps.
Part 1 — Charts	15%	Titled, business-language axis labels, colour used with intent.
Part 1 — Business recommendation	10%	Specific, data-backed, one clear action with a named store or category.
Part 2 — Ingestion & parsing	15%	JSON loaded correctly, metadata checked, DataFrame confirmed.
Part 2 — Validation function	15%	Catches all 3 issues in the corrupt file, prints a clear log, markdown explains pipeline decision.
Part 2 — Transform & summary	10%	Weekly aggregations correct, category alert uses historical baseline, generate_weekly_summary() runs cleanly.
CBIM Problem Canvas	5%	All six fields answered before any code was written.

If you are working in a group of 2–3

Role	Sections
Data Analyst	Steps 5.1–5.3 (load, store revenue, category trends) + CBIM canvas
Insights Lead	Steps 5.4–5.6 (time patterns, anomalies, business recommendation)

Pipeline Engineer	Steps 5.7–5.11 (JSON ingestion, validation, transform, summary function)

For a 2-person team: Person A takes Steps 5.1–5.6 and the CBIM canvas. Person B takes Steps 5.7–5.11. Both review and align before submitting.

What to submit	Details
Jupyter Notebook	One shared notebook. Each step's markdown credits the team member who built it.
Insight summary	1-page document (max 600 words for a group): three Part 1 findings, one Part 2 pipeline decision, one limitation, and a brief note on who owned which section.
Individual reflection	150-word note from each team member — what you contributed and what you learned from your teammate's section.

8. A Few Things Worth Flagging

What often goes wrong	What to do instead
Axis labels are column names — "store_id", "revenue"	Use business language — "Store" and "Total Revenue (₹)"
Part 1 markdown just describes the chart	Explain what it means for the business — not just what it shows
Business recommendation is vague — "improve underperforming stores"	Name the store, cite the revenue gap, state the action
Part 2 ingestion skips the metadata check	Always verify record_count matches len(df) — that is what production pipelines do
Validation function only finds one or two of the three issues	Test against all three validation rules independently — missing one is a pipeline failure
generate_weekly_summary() hardcodes numbers instead of computing them	Every number in the output should be derived from the data, not typed in
CBIM canvas filled in after the analysis	The canvas goes first — that is not a formality, it changes how you approach the problem

9. Before You Submit

Individual

- CBIM Problem Canvas completed — all six fields — before any code was written

- Part 1: All six steps present with code and markdown explanation in each
- Part 1: Every chart has a title and business-language axis labels
- Part 1: Business Recommendation names a specific store or category with a revenue figure
- Part 2: JSON loaded and metadata record count verified against actual records
- Part 2: Validation function catches all three issues in the corrupt file
- Part 2: Category alert uses the 6-month CSV baseline, not the weekly data alone
- Part 2: generate_weekly_summary() runs cleanly and produces computed (not hardcoded) numbers
- Full notebook runs end-to-end without errors — kernel restart + run all cells tested
- Insight summary submitted (max 500 words)
- Files named: [YourName]_MUC01_Notebook.ipynb and [YourName]_MUC01_Summary.docx

Group

- CBIM canvas completed together — all members contributed
- Each step's markdown credits the team member who built it
- Part 1 and Part 2 connect — the category alert in Step 5.10 references the Part 1 analysis
- Chart style is consistent across all sections — same palette, same labelling standard
- Validation function catches all three issues in the corrupt file
- generate_weekly_summary() runs cleanly on the clean JSON file
- Notebook runs end-to-end without errors — tested by a member who did not write that section
- Insight summary connects findings from both parts, not just one person's contribution
- Individual 150-word reflection submitted separately by each team member
- Files named: [TeamName]_MUC01_Notebook.ipynb and [TeamName]_MUC01_Summary.docx

---

10. Trainer Notes

This section is for the trainer — not shared with students.

- **Three files are pre-generated: the CSV (~108,000 rows), the clean JSON (4,291 records, Jun 24–30), and the corrupt JSON (same records with negative revenue at row 5, null units_sold at row 12, invalid store S99 at row 20). Share all three alongside this document.** Files ready.

- **The three intentional patterns in the CSV: top stores S01, S03, S07 / bottom stores S05, S10, S12 / Electronics declining ~25% Jan–Jun / three anomaly weeks (S05: Feb 9, S10: Apr 6, S12: Mar 16). Watch for recommendations that are vague — push students to name a specific store and a specific revenue gap.** Part 1 coaching.

- **The most common failure in Step 5.9 is checking for only one or two of the three data issues. Make this a hard criterion — all three must be caught. The markdown question about what a pipeline**

should do with corrupt data has no single right answer, but students should argue for one approach and justify it. Part 2 coaching.

- **The category_alert in Step 5.10 is the critical link — it forces students to use the historical CSV as a baseline for the weekly pipeline. Students who skip this and compute the alert only from the weekly data miss the point of the exercise. Flag this in review.** The Part 1–Part 2 connection.

- **The SSOT field is the most interesting one for this use case — there are two sources (CSV and JSON) and students need to explain how they relate. Look for students who say "the CSV is historical context, the JSON is the live signal" — that is the right mental model.** CBIM canvas.

- **Part 1 is designed for the 2.5-hour EDA session. Part 2 adds approximately 2 hours of work. With the two modules now combined, set the submission deadline 48 hours after the Data Engineering masterclass to give students time for the full pipeline build and write-up.** Timing.